

Model-Driven Software Development

Validation & Scope



Miguel Campusano

SDU Software Engineering

Previous class

Advanced topics on parsing

Summary

Abstract Syntax Tree

Parsing problems

Next:

Validation

Scope

$\text{Exp} ::= \text{INT} \mid \text{Exp} \text{ '+' } \text{Exp}$ Ambiguous

$\text{Exp} ::= \text{INT} \mid \text{INT} \text{ '+' } \text{Exp}$ Right Associative

$\text{Exp} ::= \text{INT} \mid \text{Exp} \text{ '+' } \text{INT}$ Left associative, but left recursive grammar

$\text{Exp} ::= \text{INT} \text{ ('+' INT)}^*$ Left associative with tree rewrite action

Class Description V2: math expressions

system University

entity Person

attribute name **is** String

attribute age **is** Number

require age >= 0

entity Student

 - id **is** Number

require age >= 15

entity Teacher

entity Course

attribute title **is** String

attribute ects **is** Number

require ects > 0 && ects <= 3*10

Student **is** Person

Teacher -> Person

Teacher **1--1** Course

Student ***--*** Course

S ::= 'system' ID E* R*

E ::= 'entity' ID (At|Q)*

At ::= ('attribute' | '-') ID 'is' T

Q ::= 'require' Exp

T ::= ...

R ::= I|C

I ::= ID_E ('is' | '->') ID_E

C ::= ID_E ('1-1' | '*-*') ID_E

Exp ::= ...

Parsing problems

We want a well-defined AST

It simplifies the generation (interpretation) of our code

We give some *semantics* to the AST

Ambiguity

Left recursive grammars

Associativity

Precedence

Parsing problem: Associativity

...

Exp: Number ('+' Number)*

Number: value=INT

10 + 5 + 2

10 - 5 - 2

...

Exp: Number ({Plus.left=current} '+' right=**Number**)*

Number: value=INT

Parsing problem: Precedence

```
...  
Exp: Plus  
Plus returns Expression: Term ({Plus.left=current} '+' right=Term)*  
Term returns Expression: Factor ({Mult.left = current} '*' right=Factor)*  
Factor returns Expression: Number | '(' Exp ')'  
Number: value=INT
```

10 + 5 * 3 + 1

(10 + 5) * (3 + 1)

Project: expectations

Group

Metamodel design → Solve a family of problems (topic)

Internal DSL → Good technique in a PL of your choice

External DSL syntax → Translate your internal DSL to external DSL

Code generation → Generate working code

External DSL advance → a somewhat interesting grammar

Left recursion → e.g., math exp

Name resolution → use of square brackets

Discussion

Discussion: Projects & Extensions

Design should be *interesting*

- Solve a domain-specific problem

- Language with some *behavior*

- Grammar: left-recursive and using name resolution (basic scope)

Extension

- Deeper knowledge on *something*

- Interesting* → it must be approved

Examples

- Type checkers

- Extension: new feature + testing + validation (topics combination)

- Visual programming: using blockly

Model-Driven Software Development

Validation & Scope



Miguel Campusano

SDU Software Engineering

Today

Validation

Scope

Motivation

Constraints

Class Description V3: V2 + Scope/Validation

system University

entity Person

attribute name **is** String

attribute age **is** Number

require age >= 0

entity Student

 - id **is** Number

require age >= 15

entity Teacher

entity Course

attribute title **is** String

attribute ects **is** Number

require ects > 0 && ects <= 3*10

Student **is** Person

Teacher -> Person

Teacher **1--1** Course

Student ***--*** Course

S ::= 'system' ID E* R*

E ::= 'entity' ID (At|Q)*

At ::= ('attribute' | '-') ID 'is' T

Q ::= 'require' Exp

T ::= ...

R ::= I|C

I ::= ID_E ('is' | '->') ID_E

C ::= ID_E ('1-1' | '*-*') ID_E

Exp ::= ...

Validation

A dog cannot meow

Class Description V3: Validation

system University

entity Person

attribute name **is** String

attribute age **is** Number

require age >= 0

entity Student

 - id **is** Number

require age >= 15

entity Teacher

entity Course

attribute title **is** String

attribute ects **is** Number

require ects > 0 && ects <= 3*10

Student **is** Person

Person -> Student

S ::= 'system' ID E* R*

E ::= 'entity' ID (At|Q)*

At ::= ('attribute' | '-') ID 'is' T

Q ::= 'require' Exp

T ::= ...

R ::= I|C

I ::= ID_E ('is' | '->') ID_E

C ::= ID_E ('1-1' | '*-*') ID_E

Exp ::= ...

Validation warm-up

system University

Class names should start with uppercase

entity Person

attribute name **is** String

attribute age **is** Number

require age >= 0

entity Student

 - id **is** Number

require age >= 15

entity Teacher

entity Course

attribute title **is** String

attribute ects **is** Number

require ects > 0 && ects <= 3*10

~~Student **is** Person~~

~~Person **→** Student~~

Validation: in-class exercise

system University

Self inheritance

entity Person

attribute name **is** String

attribute age **is** Number

require age >= 0

entity Student

 - id **is** Number

require age >= 15

entity Teacher

entity Course

attribute title **is** String

attribute ects **is** Number

require ects > 0 && ects <= 3*10

~~Student **is** Person~~

~~Person -> Student~~

Student -> Student

Validation: live coding

system University

Circular inheritance

entity Person

attribute name **is** String

attribute age **is** Number

require age >= 0

entity Student

 - id **is** Number

require age >= 15

entity Teacher

entity Course

attribute title **is** String

attribute ects **is** Number

require ects > 0 && ects <= 3*10

Student **is** Person

Person -> Student

Student -> Student

Validation

We cannot add every constraints into the grammar

We do not want complex specifications

It may be impossible (Context-free grammars)

A dog meows

We *reduce* the possible metamodel instances

Bettini

... do as little in the grammar and as much as possible in validation

Validation: individual extension

Validation of static properties

Class diagram: inheritance and cycles

What can/should you validate in your projects?

If chosen:

Propose validation ideas

You need to have *enough*

Type Checker: validation problem

This is *enough*

Scope

Dog x barks to dog y

Class Description V3: Scope

system University

entity Person

attribute name **is** String

attribute age **is** Number

require age >= 0

entity Student

 - id **is** Number

require age >= 15

entity Teacher

entity Course

attribute title **is** String

attribute ects **is** Number

require ects > 0 && ects <= 3*10

Student **is** Person

Teacher -> Person

Teacher **1--1** Course

Student ***--*** Course

S ::= 'system' ID E* R*

E ::= 'entity' ID (At|Q)*

At ::= ('attribute' | '-') ID 'is' T

Q ::= 'require' Exp

T ::= ...

R ::= I|C

I ::= ID_E ('is' | '->') ID_E

C ::= ID_E ('1-1' | '*-*') ID_E

Exp ::= ...

Scope warm-up

```
system University
```

```
entity Person
  attribute name is String
  attribute age is Number
  require age >= 0
```

```
entity Student
  - id is Number
  require age >= 15
```

```
entity Teacher
```

```
entity Course
  attribute title is String
  attribute ects is Number
  require ects > 0 && ects <= 3*10
```

```
Student is Person
Teacher -> Person
Teacher 1--1 Course
Student *--* Course
```

Name resolution

Without this, we cannot modify the scope

Modify default scope

Accept parents

Scope: in-class exercise

```
system University
```

```
entity Person
  attribute name is String
  attribute age is Number
  require age >= 0
```

```
entity Student
  - id is Number
  require age >= 15
```

```
entity PhDStudent
  require age >= 18
```

```
entity Course
  attribute title is String
  attribute ects is Number
  require ects > 0 && ects <= 3*10
```

```
Student is Person
PhDStudent -> Student
```

Name resolution

Without this, we cannot modify the scope

Modify default scope

Go up the inheritance graph

Scope

Linking: uses of names/IDs to the element declarations

The scope of a declaration is the part of the program where it is *visible*

Scopes can be nested

- Scope and outer (parent) scope

- Look for the declaration in the inner scope,

- If not found, up one level, and so on

Scope

Lexical Scope

Scope is determined from the lexical (static) context

Dynamic Scope

Scope is determined from the call chain

```
var x = 10;  
def funA() { return x }  
def funB() { var x = 20; return funA() }  
funB()
```

Scope in Xtext

References

`VarDecl = name=ID ' = ' Exp;`

`VarRef = name=[VarDecl];`

Default scope is the same *container*

Scope

What else can we use scope for?

Variable shadowing

In a scope, use the same variable name as a variable in an outer scope

Scope vs Validation

Forward reference

Use a symbol before its declaration

Visibility

public / private attributes

More examples?

Loose scoping, Strict validation

[PollEv.com/mica](https://poll-ev.com/mica)

Scope and Validation

How do they interact with each other?

Circular inheritance?

What happens to the scope?

Individual extension goal

What it is expected

Individual extension: expectations

Group project up to previous lecture → half of the semester

Individual extension → something about the second half

Everything about the second half → a lot of superficial things

Expert on a topic of the second half → deep knowledge on one topic

Individual extension: expectations

Individual extension

More *interesting/missing* features: depending on your projects

Anything left from the rest of the course: validation, scope, etc

Type Checker: complex validation

Modify default scope: eg: class inheritance + attribute modifiers

Visual DSL: or something more than “text”

Testing: complex enough (not covered in class)

Module system (not covered in class)

...

Individual extension: concrete topics

Complete your DSL: *Group project minimal features, how is it a complete DSL?*

Class description: missing definitions / generate different languages / ...

Scope: *Name resolution++*

Class description: inheritance, *parent* attribute reference, and modifiers

Var example: Different scopes and definitions – hoisting and var shadowing

Validation: *Reduce the metamodel instances without modifying the grammar*

Class description: expressions and type checker

Visual / Interactive DSL: *less text / connect code and data with model (traceability)*

Class description: UML-type implementation

Interactive Fiction: Use of blockly to create (visual) and display answers

Testing: *We should test our systems*

Class description: test grammar, generated text and code execution (meta-prog)

Individual extension: more *examples*

Not so concrete topics: Inspiration outside this course

Module system: *Use of multiple files and imports*

Example: University system *uses* Person system

Xtext and Blockly: *Using xtext definition file in blockly*

Code interpretation: *Extend eclipse to execute and visualize your execution*

Focus on something you *really* want to learn

Individual extension: next steps

Concrete or not, I must approve your extension

Approve: your extension has the potential to have a good grade

We can discuss together if you have an idea that needs to be scoped

Exam

I *may* ask you for your implementation

Be ready to submit (during the exam), the individual extension files

Only individual extension files, not the whole project

Summary

Summary

Scope

Validation

```
system University
```

```
entity Person
  attribute name is String
  attribute age is Number
  require age >= 0
```

```
entity Student
  - id is Number
  require age >= 15
```

```
entity PhDStudent
  require age >= 18
```

```
entity Course
  attribute title is String
  attribute ects is Number
  require ects > 0 && ects <= 3*10
```

```
Student is Person
PhDStudent -> Student
```

```
Person is PhDStudent
```

Group Project Work

Week 7

Continue working on your group project

Research, choose *and work* on individual extension